

Guía de Bypass de Integridad y Atestación de Protocolo en Entornos Multiplataforma

Este manual técnico proporciona la especificación criptográfica completa, el análisis de vulnerabilidades lógicas y el procedimiento detallado para neutralizar el handshake de atestación física de red y la validación de integridad local (HMAC) en los cargadores binarios del MMO.

Este procedimiento es general, multiplataforma (compatible con Linux y Windows) y aplicable a cualquier actualización del cliente de juego.

1. El Handshake Dinámico de Atestación (Bypass de Red)

1.1 El Flujo del Handshake

El servidor valida la legitimidad del cliente web/desktop verificando el campo "size" durante la solicitud de inicio de sesión: $\text{size} = \text{SHA-256}(\text{loginId} + \text{nonce} + K)$

Donde:

- **loginId**: Cadena alfanumérica única provista por la API web de autenticación del usuario.
- **nonce**: Cadena dinámica de desafío (challenge) enviada por el servidor en la respuesta del pre-login.
- **\$K\$**: Firma de integridad física de 32 bytes calculada localmente en la memoria RAM por el cargador original.

1.2 La Firma Constante \$K\$

Debido a que el cálculo de \$K\$ es estático (no depende de parámetros dinámicos del servidor, sino de las propiedades binarias de los archivos descargados), este valor se mantiene constante para cada versión específica del juego.

Script Para Generar \$K\$:

Este script funciona de forma automática para obtener el valor \$K\$ de cada MMO actualizado:

```
#!/usr/bin/env python3
# Devuelve SOLO la última K limpia

import struct
import zlib
import hashlib
import urllib.request

LOADER_URL = "http://cdn-ar.mundogaturro.com/juego/MMOLoader.swf"
MMO_URL = "http://cdn-ar.mundogaturro.com/juego/MMO.swf"

def download(url):
    req = urllib.request.Request(
        url,
        headers={"User-Agent": "Mozilla/5.0"}
```

```

    )
    with urllib.request.urlopen(req) as r:
        return r.read()

def parse_swf(data):
    if data[:3] == b"CWS":
        body = zlib.decompress(data[8:])
        return (
            data[:8].replace(b"CWS", b"FWS") + body,
            len(data),
            len(body) + 8
        )
    return data, len(data), len(data)

def decrypt_mmo(data, lc, ld):
    body = data[32:]

    acc = ""
    for n in (len(body), lc, ld):
        for d in str(n):
            acc += chr(int(d) + 11)

    key = hashlib.sha256(acc.encode()).digest()
    ks = hashlib.sha256(key).digest()

    return bytes(
        b ^ ks[i % 32]
        for i, b in enumerate(body)
    )

def b36(n):
    chars = "0123456789abcdefghijklmnopqrstuvwxyz"
    s = ""
    while n:
        s = chars[n % 36] + s
        n //= 36
    return s or "0"

def calc_k(mmo, lc, ld, crypted_size, mmo_comp):
    if mmo[:3] == b"CWS":
        body = zlib.decompress(mmo[8:])
        mmo = mmo[:8].replace(b"CWS", b"FWS") + body

    mmo_decomp = len(mmo)
    frag_len = mmo_decomp // 4
    frag = mmo[:frag_len]

    h = hashlib.sha256(hashlib.sha256(frag).digest()).digest()

    primes = [101, 103, 107, 109, 113, 127, 131, 137]
    MOD = 1000000007
    slots = [0, 0, 0, 0]

    for i, b in enumerate(h):
        slots[i % 4] = (
            slots[i % 4] + b * primes[i % 8]
        ) % MOD

    mask16 = (
        frag_len +

```

```

        mmo_decomp +
        ld +
        lc +
        crypted_size +
        mmo_comp
    ) & 0xFFFF

    for i in range(4):
        slots[i] = (slots[i] ^ mask16) % MOD

    base = "".join(b36(x) for x in slots)

    T = str(frag_len + ld + lc).encode()
    M = hashlib.sha256(base.encode()).digest()

    K = bytes(
        M[i] ^ T[i % len(T)]
        for i in range(len(M))
    )

    return hashlib.sha256(K).hexdigest()

loader = download(LOADER_URL)
crypted = download(MMO_URL)

_, lc, ld = parse_swf(loader)
mmo = decrypt_mmo(crypted, lc, ld)

print(
    calc_k(
        mmo,
        lc,
        ld,
        len(crypted),
        len(mmo)
    )
)

```

Cuando el CDN actualiza sus archivos, los tamaños de los binarios varían y la constante \$K\$ debe ser recalculada.

1.3 Procedimiento de Obtención de la Constante \$K\$

Para derivar la constante \$K\$ correspondiente a una actualización sin necesidad de ejecutar un depurador en memoria:

1. Descargar los binarios del CDN:

- MMOLoader . swf (Cargador comprimido)
- MMO . swf (Archivo de juego cifrado)

2. Extraer los tamaños físicos del Loader:

- loader_comp_size: Tamaño exacto en bytes del archivo MMOLoader . swf descargado (comprimido en CWS).
- loader_decomp_size: Tamaño del SWF del Loader tras ser descomprimido en formato FWS. Se obtiene leyendo la cabecera del SWF o guardándolo sin compresión

desde herramientas de edición como JPEXS.

3. Extraer los tamaños físicos del MMO:

- `crypted_mmo_size`: Tamaño exacto en bytes de `MMO.swf` en disco.
- `mmo_comp_size`: Tamaño del cuerpo comprimido del MMO descifrado (restando la cabecera HMAC de 32 bytes).
- `mmo_decomp_size`: Tamaño total del MMO descriptado y descomprimido en formato FWS (inflado con Zlib).
- `fragmentLen`: El 25% del tamaño del MMO descomprimido (`mmo_decomp_size / 4` mediante división entera).

4. Derivar la constante `$K$`:

- Calcular el doble SHA-256 del primer fragmento de tamaño `fragmentLen` del MMO descomprimido.
- Realizar una mezcla por módulo primo (`1000000007`) con las constantes de tamaño físico acumuladas:
$$\text{\texttt{\$sumaTamanios}} = \text{\texttt{\$fragmentLen}} + \text{\texttt{\$mmo_decomp_size}} + \text{\texttt{\$loader_decomp_size}} + \text{\texttt{\$loader_comp_size}} + \text{\texttt{\$crypted_mmo_size}} + \text{\texttt{\$mmo_comp_size}}$$
- Generar la máscara XOR de red utilizando:
$$\text{\texttt{\$maskValue}} = \text{\texttt{\$fragmentLen}} + \text{\texttt{\$loader_decomp_size}} + \text{\texttt{\$loader_comp_size}}$$
- Aplicar XOR del string representativo de `maskValue` sobre los bytes resultantes y generar el hash final SHA-256. El resultado de 32 bytes es la constante `$K$` estática requerida.

2. La Verificación de Integridad Física Local (HMAC del MMO)

2.1 El Crasheo al 100% de la Carga

Cuando se inyecta un archivo de juego modificado a través del cargador original, la barra de progreso se detiene al final o el proceso aborta en silencio.

Esto ocurre porque el cargador ejecuta una validación de firma criptográfica local **antes** de cargar los bytes en memoria. Si los primeros 32 bytes del archivo `MMO.swf` no coinciden con el HMAC-SHA-256 dinámico que calcula el Loader, la ejecución se cancela.

2.2 Requisitos del HMAC-SHA-256 Local

Cualquier archivo modificado que se desee inyectar debe contar con una cabecera de 32 bytes calculada específicamente sobre su tamaño actual (`$n_1$`):

- **Acumulador (`acc`)**: Se concatenan las representaciones en caracteres decimales de `$n_1$` (cuerpo modificado), `$n_2$` (peso comprimido del loader original) y `$n_3$` (peso descomprimido del loader original), aplicando un desplazamiento de +11 en base ASCII por cada dígito.

- **Clave HMAC:** SHA-256 del acumulador `acc`.
- **Mensaje:** Estructura binaria Big Endian de 12 bytes que empaqueta los tres enteros: (`n1`, `n3`, `n2`).
- **Cabecera de salida:** HMAC-SHA-256 de la clave y el mensaje. Esta firma de 32 bytes debe ser concatenada al principio del cuerpo del SWF modificado (que previamente fue encriptado mediante XOR con el keystream derivado de la clave HMAC).

3. Algoritmo de Empaquetado Automático (Python 3)

El siguiente script de Python automatiza tanto la encriptación XOR como el cálculo dinámico de la firma HMAC para cualquier SWF modificado, haciéndolo compatible de forma nativa con el Loader oficial de Mundo Gaturro:

```
#!/usr/bin/env python3
import os
import sys
import struct
import hashlib
import hmac

def build_compatible_swf(input_path, output_path):
    if not os.path.exists(input_path):
        print(f"[-] Error: Archivo {input_path} no encontrado.")
        return

    with open(input_path, "rb") as f:
        plaintext = f.read()

    # Constantes físicas requeridas
    n1 = len(plaintext)
    n2 = 98298      # loaderInfo.bytesTotal (comprimido original)
    n3 = 253358     # loaderInfo.bytes.length (descompreso original)

    print(f"[*] Empaquetando {input_path} (Cuerpo: {n1} bytes)")

    # 1. Construcción del acumulador decimal con desplazamiento (+11)
    acc = ""
    for n in (n1, n2, n3):
        for digit in str(n):
            acc += chr(int(digit) + 11)

    # 2. Derivación de Clave y Keystream
    hmac_key = hashlib.sha256(acc.encode('utf-8')).digest()
    keystream = hashlib.sha256(hmac_key).digest()

    # 3. Cálculo de la Firma de Cabecera (HMAC-SHA-256)
    hmac_message = struct.pack(">III", n1, n3, n2)
    hmac_digest = hmac.new(hmac_key, hmac_message, hashlib.sha256).digest()

    # 4. Encriptación XOR Cíclica del Cuerpo
    ciphertext = bytearray(n1)
    for i in range(n1):
        ciphertext[i] = plaintext[i] ^ keystream[i % 32]

    # 5. Concatenar cabecera HMAC + cuerpo encriptado
    final_data = hmac_digest + ciphertext
```

```

with open(output_path, "wb") as f:
    f.write(final_data)

print(f"[+] Archivo cifrado y empaquetado con éxito: {output_path}")

if __name__ == "__main__":
    if len(sys.argv) < 3:
        print("Uso: python3 encrypt.py <input.swf> <output.swf>")
    else:
        build_compatible_swf(sys.argv[1], sys.argv[2])

```

3.2 Algoritmo de Desempaquetado Automático (Python 3)

El siguiente script en Python 3 automatiza el descifrado y la descompresión del binario protegido MMO.swf, removiendo la cabecera HMAC y revirtiendo el cifrado XOR cíclico para dejar el archivo completamente limpio y listo para su análisis en descompiladores como FFDec (JPEXS):

```

#!/usr/bin/env python3
import os
import sys
import argparse
import struct
import zlib
import hashlib
import hmac

def parse_swf_sizes(swf_path):
    """
    Parsea un archivo SWF para obtener su tamaño en disco (comprimido)
    y su tamaño real una vez descomprimido en RAM.
    """
    if not os.path.exists(swf_path):
        raise FileNotFoundError(f"Loader no encontrado en: {swf_path}")

    with open(swf_path, "rb") as f:
        data = f.read()

    comp_size = len(data)
    if comp_size < 8:
        raise ValueError("El archivo SWF es demasiado pequeño o está corrupto.")

    sig = data[:3]
    # El tamaño descomprimido se guarda como entero de 32 bits Little Endian en
    los bytes 4-7
    decomp_size = struct.unpack("<I", data[4:8])[0]

    return comp_size, decomp_size

def auto_decompress_swf(data):
    """
    Detecta si el SWF está comprimido (CWS) y lo infla a formato plano (FWS)
    para facilitar su lectura directa en FFDec.
    """
    if len(data) < 8:
        return data

    sig = data[:3]
    if sig == b"CWS":
        print("[*] Detectado SWF comprimido (CWS). Descomprimiendo en
        memoria...")
        try:
            body = zlib.decompress(data[8:])
            # Reconstruir cabecera FWS (descomprimida) con la firma modificada

```

```

        header = b"FWS" + data[3:8]
        return header + body
    except zlib.error as e:
        print(f"[!] Advertencia: No se pudo descomprimir el cuerpo Zlib
({e}). Se guardará cifrado.")
    return data
def decrypt_mmo(input_path, output_path, loader_path=None):
    if not os.path.exists(input_path):
        print(f"[-] Error: Archivo de entrada {input_path} no encontrado.")
        return
    with open(input_path, "rb") as f:
        data = f.read()
    file_size = len(data)
    if file_size < 32:
        print(f"[!] Error: El archivo es demasiado corto para contener un HMAC
válido.")
        return
    # 1. Separar la cabecera HMAC de 32 bytes del cuerpo encriptado
    original_hmac = data[:32]
    ciphertext = data[32:]
    n1 = len(ciphertext)
    # 2. Obtener variables del Loader (dinámicas o fallback por defecto)
    if loader_path:
        try:
            n2, n3 = parse_swf_sizes(loader_path)
            print(f"[+] Parámetros dinámicos del Loader extraídos de
'{loader_path}':")
        except Exception as e:
            print(f"[!] Error al parsear el Loader ({e}). Usando valores por
defecto.")
            n2, n3 = 98298, 253358
    else:
        # Valores por defecto de la versión estable de Mundo Gaturro
        n2 = 98298      # loaderInfo.bytesTotal
        n3 = 253358     # loaderInfo.bytes.length
        print(f"[*] Usando parámetros de Loader por defecto (hardcoded):")

    print(f"    - n1 (Cuerpo MMO): {n1} bytes")
    print(f"    - n2 (Loader Comprimido): {n2} bytes")
    print(f"    - n3 (Loader Descomprimido): {n3} bytes")
    # 3. Derivación de la clave a través del acumulador decimal (+11 ASCII)
    acc = ""
    for n in (n1, n2, n3):
        for digit in str(n):
            acc += chr(int(digit) + 11)
    hmac_key = hashlib.sha256(acc.encode('utf-8')).digest()
    keystream = hashlib.sha256(hmac_key).digest()
    # 4. Validación estricta del HMAC (Seguridad e Integridad)
    expected_message = struct.pack(">III", n1, n3, n2)
    computed_hmac = hmac.new(hmac_key, expected_message,
hashlib.sha256).digest()
    if computed_hmac != original_hmac:
        print(f"[!] ADVERTENCIA: La firma HMAC calculada no coincide con la
original.")
        print(f"    - Esperada: {computed_hmac.hex()}")
        print(f"    - Recibida: {original_hmac.hex()}")
        print(f"    [?] Esto suele significar que los pesos del Loader (n2, n3)
no son los correctos.")
    else:
        print(f"[+] Integridad verificada: ¡Firma HMAC válida!")
    # 5. Descifrado XOR Cíclico en memoria
    print(f"[*] Descifrando cuerpo binario...")
    plaintext = bytearray(n1)
    for i in range(n1):

```

```

        plaintext[i] = ciphertext[i] ^ keystream[i % 32]
# 6. Descompresión automática e inyección en disco
final_swf = auto_decompress_swf(bytes(plaintext))
with open(output_path, "wb") as f:
    f.write(final_swf)
magic = final_swf[:3].decode('utf-8', errors='ignore')
print(f"[+++] PROCESO COMPLETADO: Guardado en '{output_path}'")
print(f"        Firma final: {magic} | Tamaño final: {len(final_swf)} bytes
(Listo para FFDec)")
if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description="Extractor y descifrador inteligente de MMO.swf para Mundo
Gaturro."
    )
    parser.add_argument("input", help="Ruta al MMO.swf cifrado original.")
    parser.add_argument("output", help="Ruta de destino del SWF descifrado
final.")
    parser.add_argument(
        "-l", "--loader",
        help="Ruta al MMOLoader.swf actual del CDN para extraer parámetros
automáticamente (opcional).",
        default=None
    )
    args = parser.parse_args()
    decrypt_mmo(args.input, args.output, args.loader)

```

4. Redirección de Red e Intercepción Dinámica (Linux & Windows)

Para jugar de manera transparente utilizando un cliente de juego modificado, se debe implementar una técnica de redirección de tráfico local en dos fases:

1. **Redirección Estática:** Desviar la petición del CDN (cdn-ar.mundogaturro.com/juego/MMO*.swf) para servir tu SWF empaquetado localmente.
2. **Modificación en Runtime:** Interceptar el paquete socket de login para leer de manera transparente el `loginId` y el `nonce`, calcular la atestación física usando la constante `$K$` y reinyectar el campo `size` correcto antes de que llegue al servidor.

4.1 Método A: Intercepción Dinámica en Linux

Utiliza un proxy TCP/Websockets basado en Python que automatiza todo el proceso:

1. **Configurar el Proxy Websocket (proxy_server.py):** Levanta un proxy local que escuche las peticiones websocket del juego. Al capturar la respuesta del servidor con el `nonce` y la solicitud de login del cliente con el `loginId`:
 - Lee los campos en caliente.
 - Calcula de forma automática el SHA-256 con el valor de tu constante `$K$`.
 - Reescribe el campo `size` del JSON de login.
 - Enruta el payload corregido al servidor.

2. Iniciar el Bypass:

```
gaturro-desktop --proxy-server="127.0.0.1:8080" --ignore-certificate-errors --no-sandbox
```

4.2 Método B: Redirección mediante el archivo Hosts + Proxy Local WINDOWS

El archivo `hosts` de Windows funciona como una agenda o directorio telefónico local: permite asignar nombres de dominio a direcciones IP manualmente, obligando a tu computadora a redirigir el tráfico hacia donde vos decidas **antes** de consultar a los servidores DNS tradicionales.

Paso 1 — Editar el archivo hosts

1. Abrió el **Bloc de notas** como administrador:

- Presioná **Win**, escribí **Bloc de notas**, clic derecho → **Ejecutar como administrador**.

2. Desde el Bloc de notas, andá a **Archivo** → **Abrir** y navegá a:

```
C:\Windows\System32\drivers\etc
```

Cambiá el filtro a **Todos los archivos (*.*)**, seleccioná **hosts** y abrílo.

3. Al final del archivo, agregá estas dos líneas:

```
127.0.0.1    juegosg07.mundogaturro.com
127.0.0.1    juegosg01.mundogaturro.com
```

4. Guardá con **Ctrl + S**. Si da error de permisos, repetí desde el paso 1 asegurándote de abrir el Bloc de notas como administrador.

5. Limpiá la caché DNS para que el cambio tome efecto:

```
ipconfig /flushdns
```

1.1 Ejecutar Proxy Para Calcular Size E Inyectarlo

El siguiente script en Python 3 es un proxy especialmente para generar el size en milisegundos sin tu hacer más nada que solo ejecutarlo y dejarlo en segundo plano:

```
import asyncio
import struct
import sys
import os
import json
import hashlib

sys.path.append(os.getcwd())
try:
    from mambo_protocol import MamboProtocol
except ImportError:
    print("[!] Error: No se encontró mambo_protocol.py.")
    sys.exit(1)
```

[illegible]

```

        buf.extend(nonce.encode('ascii'))
        buf.extend(K_BYTES)

        size_hex =
hashlib.sha256(buf).digest().hex()
        mobj["size"] = "HEX:" + size_hex
        full_packet =
MamboProtocol.encode_packet(packet["obj"], packet.get("header"))
        print(f"[+] size inyectado: {size_hex}")
    else:
        print("[!] Login detectado pero falta hash o
nonce.")

        dst_writer.write(full_packet)
        await dst_writer.drain()
        header = await src_reader.readexactly(5)

    except Exception as e:
        print(f"[!] forward {direction} cerrado: {e}")
    finally:
        dst_writer.close()

    await asyncio.gather(
        forward(reader, remote_writer, "C->S", peek_data),
        forward(remote_reader, writer, "S->C", b"")
    )

    except Exception as e:
        print(f"[!] Conexión fallida: {e}")
    finally:
        writer.close()

async def main():
    proxy = SimpleProxy()
    server = await asyncio.start_server(proxy.handle_connection, '0.0.0.0',
9899)
    server2 = await asyncio.start_server(proxy.handle_connection, '0.0.0.0',
9898)
    print("[*] Proxy escuchando en :9898 y :9899")
    await asyncio.gather(server.serve_forever(), server2.serve_forever())

if __name__ == "__main__":
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        pass

```

1.2 Script Para Deserializar Procolo Mambos

El siguiente script en Python 3 es un deserializador de el protocolo Mambos fundamental para que el proxy funcione:

```

import struct
import binascii
import zlib
import logging

log = logging.getLogger("MamboProtocol")

class MamboProtocol:
    """

```

Mambo Protocol SDK (Mundo Gaturro WebSocket Binary Protocol).

Data Types:

```
'n' (0x6E) : Int32 (4 bytes)
'l' (0x6C) : Float32 (4 bytes)
't' (0x74) : String (4 bytes length + UTF-8 bytes)
'o' (0x6F) : Boolean (1 byte, 0 or 1)
'j' (0x6A) : Object (4 bytes property count + [type][key_length]
[key_string][value]...)
'J' (0x4A) : Mixed Array (4 bytes count + [type][value]...)
'N' (0x4E) : Int32 Array (4 bytes count + [int32]...)
'T' (0x54) : String Array (4 bytes count + [4 bytes length + UTF-8
bytes]...)
'L' (0x4C) : Float32 Array (4 bytes count + [float32]...)
'O' (0x4F) : Boolean Array (4 bytes count + [bool]...)
'Z' (0x5A) : ByteArray/Zlib Blob (4 bytes length + bytes)
0x00 / 'z' : Null/None (no payload)
""
```

@staticmethod

```
def decode_packet(data: bytes) -> dict:
    if len(data) < 5:
        return {"raw": data, "error": "Packet too short"}

    header = data[0]
    length = struct.unpack(">I", data[1:5])[0]
    payload = data[5:]

    result = {
        "header": header,
        "length": length,
        "raw_payload": payload,
        "obj": None,
    }

    if header == 3: # Main Mambo Object payload
        try:
            decoded_obj, _ = MamboProtocol.read_mobject(payload, 0)
            result["obj"] = decoded_obj
        except Exception as e:
            log.error(f"Error decoding packet: {e}")

    return result
```

@staticmethod

```
def encode_packet(data_dict: dict, header_type: int = 3) -> bytes:
    payload = MamboProtocol.write_mobject(data_dict)
    packet = bytearray([header_type])
    packet.extend(struct.pack(">I", len(payload)))
    packet.extend(payload)
    return bytes(packet)
```

@staticmethod

```
def read_mobject(data: bytes, offset: int = 0):
    if offset + 4 > len(data):
        return {}, offset

    count = struct.unpack(">I", data[offset : offset + 4])[0]
    offset += 4

    obj = {}
    for _ in range(count):
        if offset + 1 > len(data):
            break
```

```

        type_char = chr(data[offset])
        offset += 1

        if offset + 4 > len(data):
            break

        key_len = struct.unpack(">I", data[offset : offset + 4])[0]
        offset += 4

        key = data[offset : offset + key_len].decode("utf-8",
errors="ignore")
        offset += key_len

        value, offset = MamboProtocol.read_value(data, offset, type_char)
        obj[key] = value

    return obj, offset

@staticmethod
def read_value(data: bytes, offset: int, type_char: str):
    try:
        # NULL
        if type_char == "\x00" or type_char == "z":
            return None, offset

        # INT32
        if type_char == "n":
            return struct.unpack(">i", data[offset:offset+4])[0], offset + 4

        # FLOAT32
        if type_char == "l":
            return struct.unpack(">f", data[offset:offset+4])[0], offset + 4

        # STRING
        if type_char == "t":
            l = struct.unpack(">I", data[offset:offset+4])[0]
            offset += 4
            s = data[offset:offset+l].decode("utf-8", errors="ignore")
            offset += l
            return s, offset

        # BOOLEAN
        if type_char == "o":
            return data[offset] != 0, offset + 1

        # OBJECT
        if type_char == "j":
            obj, offset = MamboProtocol.read_mobject(data, offset)
            return obj, offset

        # MIXED ARRAY
        if type_char == "J":
            count = struct.unpack(">I", data[offset:offset+4])[0]
            offset += 4
            arr = []
            for _ in range(count):
                et = chr(data[offset])
                offset += 1
                val, offset = MamboProtocol.read_value(data, offset, et)
                arr.append(val)
            return arr, offset

        # INT ARRAY

```

```

if type_char == "N":
    count = struct.unpack(">I", data[offset:offset+4])[0]
    offset += 4
    arr = []
    for _ in range(count):
        val = struct.unpack(">i", data[offset:offset+4])[0]
        offset += 4
        arr.append(val)
    return arr, offset

# STRING ARRAY
if type_char == "T":
    count = struct.unpack(">I", data[offset:offset+4])[0]
    offset += 4
    arr = []
    for _ in range(count):
        sl = struct.unpack(">I", data[offset:offset+4])[0]
        offset += 4
        s = data[offset:offset+sl].decode("utf-8", errors="ignore")
        offset += sl
        arr.append(s)
    return arr, offset

# FLOAT ARRAY
if type_char == "L":
    count = struct.unpack(">I", data[offset:offset+4])[0]
    offset += 4
    arr = []
    for _ in range(count):
        val = struct.unpack(">f", data[offset:offset+4])[0]
        offset += 4
        arr.append(val)
    return arr, offset

# BOOLEAN ARRAY
if type_char == "O":
    count = struct.unpack(">I", data[offset:offset+4])[0]
    offset += 4
    arr = []
    for _ in range(count):
        arr.append(data[offset] != 0)
        offset += 1
    return arr, offset

# BYTEARRAY / ZLIB BLOB
if type_char == "Z":
    l = struct.unpack(">I", data[offset:offset+4])[0]
    offset += 4
    blob = data[offset:offset+l]
    offset += l

    # Accept all valid zlib headers: 0x78 0x01, 0x78 0x5E, 0x78
0x9C, 0x78 0xDA
    if len(blob) >= 2 and blob[0] == 0x78 and blob[1] in (0x01,
0x5E, 0x9C, 0xDA):
        try:
            dec = zlib.decompress(blob)
            try:
                # Return with ZLIB: prefix so encoder preserves the
Z type
                return "ZLIB:" + dec.decode("utf-8",
errors="ignore"), offset
            except:
                return "HEX:" + blob.hex(), offset

```

```

        except:
            pass
        return "HEX:" + blob.hex(), offset

    # UNKNOWN TYPE CATCH-ALL
    ctx_start = max(0, offset - 32)
    ctx_end = min(len(data), offset + 32)
    ctx = data[ctx_start:ctx_end]
    hex_dump = " ".join([f"{b:02x}" for b in ctx])

    hex_type = f"{ord(type_char):02x}" if isinstance(type_char, str) and
len(type_char) == 1 else "unknown"
    log.error(f"===== TIPO DESCONOCIDO '{type_char}' (hex: {hex_type})
EN OFFSET {offset} =====")
    log.error(f"CONTEXTO HEX (-32 a +32): {hex_dump}")

    return None, offset

except Exception as e:
    log.error(f"Error parsing type {type_char} @ {offset}: {e}")

    return None, offset

@staticmethod
def write_mobject(obj: dict) -> bytearray:
    buf = bytearray(struct.pack(">I", len(obj)))
    for k, v in obj.items():
        # Override seguro para 'check' para preservar su tipo Z / Zlib Blob
original
        if k == "check" and isinstance(v, str) and not v.startswith("ZLIB:")
and not v.startswith("HEX:"):
            v = f"ZLIB:{v}"

        tc = MamboProtocol.infer_type(v)
        buf.append(ord(tc))
        kb = k.encode("utf-8")
        buf.extend(struct.pack(">I", len(kb)))
        buf.extend(kb)
        buf.extend(MamboProtocol.write_value(v, tc))
    return buf

@staticmethod
def infer_type(v) -> str:
    if v is None:
        return "\x00"
    if isinstance(v, bool):
        return "o"
    if isinstance(v, int):
        return "n"
    if isinstance(v, float):
        return "l"
    if isinstance(v, str):
        if v.startswith("HEX:") or v.startswith("ZLIB:"):
            return "Z"
        return "t"
    if isinstance(v, dict):
        return "j"
    if isinstance(v, list):
        if not v:
            return "J"
        if isinstance(v[0], dict):
            return "J"
        if isinstance(v[0], str):
            return "T"

```

```

        if isinstance(v[0], int):
            return "N"
        if isinstance(v[0], float):
            return "L"
        if isinstance(v[0], bool):
            return "O"
        return "J"
    return "t"

@staticmethod
def write_value(v, tc: str) -> bytes:
    if tc == "\x00" or tc == "z":
        return b""
    if tc == "n":
        return struct.pack(">i", int(v))
    if tc == "l":
        return struct.pack(">f", float(v))
    if tc == "t":
        vb = str(v).encode("utf-8")
        return struct.pack(">I", len(vb)) + vb
    if tc == "o":
        return bytes([1 if v else 0])
    if tc == "j":
        return MamboProtocol.write_mobject(v)
    if tc == "J":
        b = struct.pack(">I", len(v))
        for x in v:
            etc = MamboProtocol.infer_type(x)
            b += etc.encode("utf-8") + MamboProtocol.write_value(x, etc)
        return b
    if tc == "T":
        b = struct.pack(">I", len(v))
        for x in v:
            xb = str(x).encode("utf-8")
            b += struct.pack(">I", len(xb)) + xb
        return b
    if tc == "N":
        b = struct.pack(">I", len(v))
        for x in v:
            b += struct.pack(">i", int(x))
        return b
    if tc == "L":
        b = struct.pack(">I", len(v))
        for x in v:
            b += struct.pack(">f", float(x))
        return b
    if tc == "O":
        b = struct.pack(">I", len(v))
        for x in v:
            b += bytes([1 if x else 0])
        return b
    if tc == "Z":
        if str(v).startswith("HEX:"):
            raw_bytes = binascii.unhexlify(str(v)[4:])
        elif str(v).startswith("ZLIB:"):
            # Forzar level=9 para emular Flash ByteArray.compress() (genera
0x78 0xDA)
            raw_bytes = zlib.compress(str(v)[5:].encode("utf-8"), level=9)
        else:
            raw_bytes = str(v).encode("utf-8")
        return struct.pack(">I", len(raw_bytes)) + raw_bytes
    return b""

```


Paso 2 — Cómo funciona la redirección

Cuando el juego intente conectarse a `juegosg07` o `juegosg01`, en lugar de llegar al servidor real, va a resolver a `127.0.0.1` (tu propia máquina). A partir de ahí:

- Arrancás tu servidor/proxy local escuchando en el **puerto 9899**.
- El juego, al ser redirigido a `127.0.0.1`, se conecta a ese puerto y **tu proxy recibe la conexión**.
- Todo lo que el cliente envía pasa por el proxy, que lo reenvía al servidor real, y viceversa.
- Podés **filtrar paquetes** en cualquier dirección (cliente → servidor o servidor → cliente) y decidir qué dejás pasar, qué modificás o qué descartás.

Paso 3 — Resultado

Una vez que el proxy esté escuchando los sockets WSS de esos servidores, la redirección opera de forma automática: toda la comunicación del juego fluye por tu proxy y podés inspeccionar los paquetes en tiempo real.

Para revertir, eliminá o comentá las líneas en `hosts` (poniendo `#` al inicio) y volvé a ejecutar `ipconfig /flushdns`.